

A decorative illustration on the left side of the page. It features a stylized plant with green leaves growing out of a grey database cylinder. The cylinder has a 'python powered' logo and the word 'JSON' on it. A large blue arrow points downwards from the cylinder. At the bottom, there are several green folders and a document icon.

Rapport

Du projet

SpaceInvaders

Auteurs

Par Bacar Houssam

Et Benard Karl

L2 INFORMATIQUE
2021-2022

Sommaire

Introduction.....	3
SpaceInvaders.....	3
Game.....	4
Fleet.....	5
Alien.....	6
Defender.....	8
Bullet.....	8
Score.....	9
Résultat.....	9
Affichage.....	9
Conclusion	10

Introduction

SpacInvaders est un jeu programmé à l'aide du langage de programmation orienté objet Python 3. Le but du jeu est d'éliminer tous les aliens qui se présentent. Le défenseur est celui qui devra éliminer les aliens afin de remporter la partie. Le défenseur élimine les aliens grâce à des balles rouges dans le jeu. La programmation de ce jeu consiste à manipuler des objets et donc la problématique que j'ai émise est comment manipuler les points techniques et les objets en python pour programmer le jeu SpacInvaders. En premier lieu, nous allons présenter les objets et leur points techniques. Notons que dans ce rapport, pour parler de classe, nous parlerons plutôt d'objet, cela sous-entendra le mot classe implicitement.

Tout d'abord, je tiens à présenter les classes qu'on utilise pour faire le jeu Space Invaders. Donc ici, nous avons un objet SpacInvaders qui va lancer le jeu avec un titre donné et une frame pour préparer la zone à laquelle nous allons travailler. Nous allons programmer le jeu à l'aide du module tkinter (Python2.x) ou Tkinter (Python3.x) qui d'ailleurs est emprunté à tcl (Tool Command Language).

La zone qui va nous permettre de dessiner s'appelle le Canvas, et va nous servir principalement pour dessiner les objets du jeu.

SpacInvaders

L'objet SpacInvaders est constitué de deux méthodes :

- `__init__` : qui permet de construire l'objet en question et initialiser les attribut de l'objet
- `play` : qui va lancer la boucle de la fenetre principale et ainsi que la méthode `start_animation` de l'objet Game.

C'est cette classe là qui va nous servir pour lancer le jeu de la façon la plus simple.

Game

L'objet Game est constitué de divers méthodes et attributs nous permettant de lancer les différents partie du jeu :

- `__init__` : permet d'instancier des objets du jeu, de créer une instance de Canvas, d'installer les afficher, et de les installer s'il le faut.
- `start_animation` : permet d'amorcer la méthode d'animation grâce a une autre méthode de Canvas « `after` » qui prend en paramètre le temps d'attente avant laquelle devra être exécutée une méthode et l'autre paramètre est justement cette méthode-ci.
- `animation` : permet de faire une animation en boucle et assure les mouvements des objets dans le Canvas.
- `move_bullets` : permet de faire bouger les balles que tir un tireur, ici le défenseur.
- `move.aliens_fleet` : permet de faire bouger une flotte d'aliens
- `press` : permet la gestion des touche de clavier pour le contrôle du défenseur.
- `gestion_de_score` : permet d'enregistrer un score dans un fichier afin d'y avoir accès plus tard.

Cet objet est très important car il nous permet de créer et d'ajouter des instances d'objets à notre jeu ainsi il est possible de manipuler les objets aisément. De plus, cette classe permet de faire des affichages de score ainsi que la création d'un profil de joueur.

Fleet

La classe Fleet est la classe qui permet d'afficher la flotte et les événements qui y seront attachés (déplacements, collisions). Elle est composée de plusieurs méthodes qui sont :

- `__init__` : initialise les attributs de la classe dont :
 - `aliens_lines` : Nombre de ligne
 - `aliens_columns` : Nombre de colonne
 - `aliens_inner_gap` : L'espacement intérieur entre chaque Alien adjacent
 - `alien_x_delta` : Correspond à combien de pixels la flotte se déplace horizontalement
 - `alien_y_delta` : Correspond à combien de pixels la flotte se déplace verticalement
 - `fleet_size` : Taille de la flotte
 - `aliens_fleet` : initialise une collection qui contient les objets Alien
 - `alienTag = 'flotte'` : initialise un Tag c'est-à-dire un identifiant associé à la flotte
 - `pim` : Initialisé par l'image associée à la flotte qui sera affiché, l'image est interprété par la classe du module tk, notamment par la syntaxe `"tk.PhotoImage(file='alien.gif')"`



- `install_in` : installe la flotte d'Alien en ajoutant chaque Alien dans la collection de la flotte d'Alien :
boucle imbriquée : une boucle imbriquée ajoute un Alien à chaque case d'une ligne et une fois la ligne complète la boucle fait la même sur la prochaine ligne et ce jusqu'à ce que chaque ligne soit pleine
- `get_width` et `get_height`: Prélèvent les nouvelles coordonnées de la flotte
- `move_in`: Permet le déplacement de la flotte. De plus, le Defender perd la partie lorsque la flotte d'alien atteint la zone de déplacement du Defender



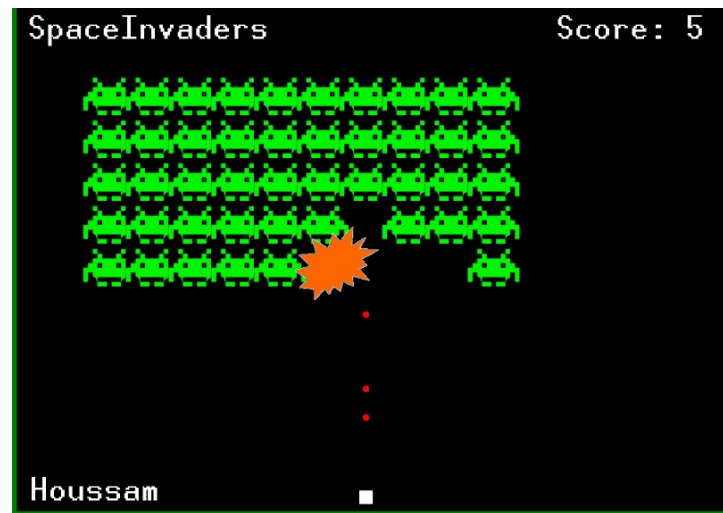
- `manage_touched_alien_by`: Gère les collisions

Au fur et à mesure de l'élimination des aliens, la flotte perd sa taille originelle, ainsi il est possible d'éliminer tous les aliens sans contraintes supplémentaire. L'objet `Fleet` est initialisé après l'attribution de la frame à l'objet `Game`. L'objet `Fleet` est par conséquent, le premier à être dessiné dans le `Canvas`. La classe `Affichage` permet d'afficher des informations supplémentaire au lancement du jeu. Cette classe n'est ici pas vraiment utilisée. Il permet d'afficher les scores, demander le nom du joueur. Et pour amélioration, permettre de cliquer sur les scores des parties précédentes.

Alien

L'Objet `Alien` permet de représenter un alien dans le jeu et l'ennemi du défenseur. Cet objet est initialisé avec la création d'une flotte d'alien. L'alien possède des attributs d'identification et un attribut de vie. L'objet traité ici est aussi composé de différentes méthodes :

- `__init__` : permet de construire l'objet en question, d'initialiser l'image de l'explosion de l'Alien
- `touched_by` : Permet de mettre en place les techniques de traitement d'une collision. La collision se fait dans cette version du jeu, entre un objet `Bullet` et l'Alien. Lorsque l'Alien est touché par un bullet celui-ci disparaît de l'écran (`self.alive` de `Alien` passe à `False`). Cela permet aussi d'afficher une explosion pendant quelques millisecondes et incrémenter le score. Dans le cas où l'Alien est touché, celui-ci est supprimé de la collection de flotte d'Alien.



- De plus, le Defender gagne lorsque son score atteint 50 points.



- `install_in` : Permet d'installer L'Alien dans le Canvas, l'alien avec une image pour le représenter.
- `move_in`: Permet de déplacer l'alien avec les coordonnées indiquées en paramètre grâce à la méthode `move` de Canvas.

L'Alien est installé dans le Canvas grâce à la classe `Fleet` pour dessiner une flotte d'Alien.

Defender

L'Objet Defender correspond au héros du jeu celui qui dans la version minimale du jeu tir, élimine tous les aliens du jeu et gagne une fois sa mission bien effectuée. L'objet Defender est composée d'attributs et de méthodes permettant de le gérer :

- `__init__` : permet de construire l'objet Defender et d'initialiser une collection des boulettes qu'il tire.
- `install_in` : Permet d'installer le Defender représenté par un carré au centre du Canvas
- `move_in` : Déplace le Defender dans le Canvas et garde aussi le Defender dans le Canvas
- `fire`: Tant que le Defender peut tirer, on crée une boulette qu'on installe dans le Canvas et on ajoute la boulette dans la collection des boulettes tirées du Defender

Le Defender est aussi le personnage principale. Dans le jeu, ce personnage, même après avoir gagné, il lui reste toujours des boulettes auxquelles il peut utiliser et réutiliser.

Difficulté ajoutée :

Le Defender ne peut tirer qu'en restant immobile.

Bullet

La classe Bullet est la classe qui permet d'afficher les projectiles tirés par l'objet Defender. Elle se fait par trois méthodes qui sont :

`__init__` : initialise les attributs de la classe dont :

radius : le rayon du projectile

color : la couleur du projectile

speed : la vitesse du projectile

id = None : l'id est None car il n'est pas encore instancié

shooter : l'objet qui tire

`install_in` : installe l'objet de la manière suivante :

À partir des coordonnées du shooter récupérer grâce `bbox`, les nouvelles coordonnées du projectile sont recalculées. Et ainsi, le projectile est dessiné (cercle rouge).

`move_in` : La boulette se déplace vers le haut (pour le Defender) tant qu'elle est dans le Canvas sinon elle est supprimée de la collection des boulettes tirés par le tireur.

Score

La classe Score permet de gérer un score. Elle est, en effet, composée d'attributs et de méthode majoritairement des accesseurs (des getters et setteurs) pour modifier les données d'objet Score. Cette classe est utilisé pour sauvegarder le score et le nom du joueur d'une partie de jeu dans un fichier json.

Résultat

La classe Résultat Permet de gérer une collection de score. Je présente ici la classe Resultat avec quelques exemples d'utilisation. Notamment, il permet :

- l' importation de fichier:
 - Avec directement le modèle Resultat (matérialisation)
 - Resultat().fromFile("monfichier.json")
- l' exportation de fichier au format json:
 - Avec instantiation du modèle Resultat (sérialisation)
 - res = Resultat()
 - res.toFile("monfichier.json")

Affichage

Affichage est un objet qui pourrait nous permettre de gérer certaines parties d'affichage du jeu. Il est constitué de méthode permettant de mettre en place les affichages de labels, des inputs, mais aussi de détruire les objets qu'elle affiche dans le jeu.

Cette classe permet d'afficher une bannière pour indiquer le nom, les scores précédents, le lancement du jeu

`__init__`: Permet d'initialiser les attributs de l'objet

`getUID`: Retourne l'id de la bannière affichée à l'écran

`setLabel`: Permet d'afficher le nom du joueur avec un label

`setName`: Permet d'entrer le nom du joueur

`setBan`: Affiche la bannière SpaceInvaders

`destroyALL`: détruit tous les objets de l'objet Affichage

`creation_cadre`: Permet de créer un cadre pour la zone d'affichage des scores et de l'entrée du nom du joueur

Bien que cette classe soit implémentée dans le jeu. Elle n'y est toutefois pas utilisé en raison d'un manque de temps pour construire l'ensemble des méthodes qui doivent la composer.

Conclusion

La programmation orienté objet nous a permis de coder le jeu SpaceInvaders. Certaines parties techniques étaient bien difficiles à gérer. L'ensemble de la programmation s'est assez bien déroulé. Dans les améliorations, l'IA nous auraient permis tirer plusieurs boulettes à un bloc d'aliens. Il reste essentiel pour notre jeu de continuer son amélioration dans le future.